

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 12 · LECTURE NOTES

Post-Quantum Cryptography

Assessed proportionately.

MODULE OUTCOMES

- State precisely what a quantum computer breaks.
- Explain why symmetric cryptography is largely fine.
- Explain the harvest-now-decrypt-later argument.
- Name the standardised schemes and their costs.
- Plan a migration proportionate to your threat model.

1 What actually breaks

Primitive	The impact	The response
RSA, Diffie–Hellman, ECDH, ECDSA, Ed25519	Broken — Shor's algorithm solves factoring and discrete log in polynomial time.	Replace (§3).
AES-128	Grover's algorithm: roughly 2^{64} work rather than 2^{128}.	Use AES-256. A parameter change, not a redesign.
AES-256	Grover's: roughly 2^{128} — which remains entirely infeasible.	Nothing.
SHA-256	Collision search improves modestly , and not dramatically.	Nothing, or move to SHA-384 for very long-lived commitments.
HMAC, AEAD constructions	Inherit the symmetric situation above — essentially fine.	Nothing.

SHOR'S ALGORITHM BREAKS THE STRUCTURE; GROVER'S ONLY SEARCHES FASTER

Shor's algorithm factors integers and computes discrete logarithms in polynomial time — which destroys RSA and every elliptic-curve scheme completely rather than merely weakening them, because it exploits the algebraic structure those schemes are built on.

Grover's algorithm gives a square-root speedup on unstructured search — so an n -bit key provides about $n/2$ bits of security against it, which is answered completely and cheaply by doubling n . **And the speedup does not parallelise well**, which makes even that estimate conservative.

So symmetric cryptography needs a parameter change and public-key cryptography needs replacement — a qualitative difference, and the entire basis of a proportionate response. The common framing of 'quantum breaks encryption' collapses this distinction and produces badly allocated effort.

And the post-quantum schemes rest on different mathematical problems rather than on better algorithms — lattice problems and error-correcting codes, for which **no quantum advantage is currently known** (Module 06 §2's fourth row). **Which is a weaker guarantee than it sounds:** 'no known quantum algorithm' is the same kind of claim as 'no known classical algorithm' was for factoring, and §4's hybrid recommendation follows from taking that seriously.

2 The timing question

HARVEST NOW, DECRYPT LATER IS THE ARGUMENT THAT JUSTIFIES ACTION TODAY

An adversary can record encrypted traffic today and decrypt it whenever a sufficiently capable machine exists — storage is cheap and patience is free — so **the relevant question is how long your data must remain confidential**, rather than when the machine arrives.

Which makes this a data-classification question rather than a cryptography question. A session token that expires in an hour is entirely unaffected; medical records, genomic data, state secrets, and identity documents have confidentiality requirements measured in decades and are affected now.

And the honest position on timing is that nobody knows. Current machines are many orders of magnitude short of what Shor's algorithm requires at cryptographic scale, and **the remaining engineering gap is large while the progress is real** — both halves of that are true, and any confident date in either direction is not supported.

So: migrate key exchange first, because key exchange is precisely where harvesting applies — and signatures are considerably less urgent, because a signature forged in 2040 on a document from 2026 is a much smaller problem than that document's contents being read. That asymmetry is the single most useful planning insight in this module.

- **1 · Key exchange protecting long-lived confidential data** — the only genuinely urgent case, and hybrid deployment addresses it today with available software.

- **2 · Long-lived signing keys and roots of trust**, whose validity extends over decades — **firmware signing keys especially**, since a device shipped now may be in service in twenty years and cannot have its root replaced.
- **3 · Ordinary TLS**, which your platform and your browser vendors will migrate for you — and where **your main job is to not obstruct it** by pinning algorithms or inspecting traffic with middleboxes that cannot be updated.
- **4 · Everything else**, on the normal upgrade cycle, as libraries and platforms deliver it.
- **And cryptographic agility is the real deliverable: a system that can change algorithm without being redesigned** — which is Module 10 §3's key identifier argument applied to algorithm selection, and is the same design lesson for the third time. **Agility is worth more than any specific algorithm choice**, because **the next transition will also happen**, and the ability to make it is the durable asset.

3 The schemes

KEY ENCAPSULATION

ML-KEM (Kyber) the standard choice.
 public key ~1.2 kB, ciphertext ~1.1 kB,
 against X25519's 32 bytes each.
 Computationally fast; the size is the cost.

SIGNATURES

ML-DSA (Dilithium) the general-purpose choice.
 signature ~2.4 kB against Ed25519's 64 bytes.
 SLH-DSA (SPHINCS+) hash-based, very conservative
 assumptions, substantially larger and slower.
 For firmware and roots of trust, where the
 conservatism is worth the cost.

THE COST IS SIZE, NOT SPEED

which matters for constrained links, embedded
 devices, certificate chains, and anything that has
 to fit inside a single packet.

AND DEPLOY HYBRID: classical plus post-quantum together,
 secure if EITHER holds. The new schemes are newer and
 have had far less cryptanalysis.

Hybrid is the right default during a transition. It hedges against the new scheme being broken, which has already happened to several candidates — including some that reached late rounds of the standardisation process and were then broken by classical attacks, in one case on a laptop in under an hour. **That history is the strongest argument for hybrid**, and it costs only bandwidth.

4 Proportionality

WHAT A PROPORTIONATE RESPONSE LOOKS LIKE

Inventory where you use public-key cryptography — which most organisations cannot currently answer, and which is therefore the actual first step rather than any algorithm decision (CSCE 701 Module 08 §3's inventory argument, arriving in a second context).

Classify your data by how long it must remain confidential, and act on the long-lived portion — which for most organisations is a small and identifiable fraction of everything they hold.

Enable hybrid key exchange where your platform offers it — which is increasingly a configuration change rather than a project, and is already deployed by default in several major browsers and TLS libraries.

And build agility rather than merely switching algorithms: algorithm identifiers in every stored format, no hardcoded key or signature sizes, no assumptions that a public key fits in 32 bytes, and a procedure for changing that you have actually executed — which is what you will need again, for the transition after this one.

- **Do not use a non-standardised scheme.** Several candidates were broken during the selection process, some spectacularly and some very late — which is exactly what a standardisation process is for.
- **Do not abandon classical cryptography. Hybrid, not replacement** — the post-quantum schemes have had a small fraction of the cryptanalytic attention that RSA and elliptic curves have absorbed over forty years.
- **Do not treat it as urgent for short-lived data**, which is the great majority of data, and where the same effort spent on §4's inventory or on CSCE 701's identity controls buys considerably more security.
- **Do not let it displace the work that actually matters. No organisation has yet been compromised by a quantum computer, and a great many have been compromised by a leaked credential or an unpatched dependency** (CSCE 701 Module 13 §4's ordering).
- **And do not claim quantum resistance you have not deployed** — which is Module 13's subject and is already a common marketing overclaim. **The last two points are the honest proportionality position: this is a real problem on a long timescale, and it is not your most likely cause of a breach this year** — and holding both of those at once is the judgement this module is trying to develop.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
NIST Post-Quantum Cryptography project (free)	§3's standards and the selection rationale — free, and considerably more measured than most commentary on the subject.
NSA CNSA 2.0 and the NIST migration guidance (free)	§2's timelines as published by parties who must plan them — useful as a datum rather than a prediction.
Bernstein & Lange — Post-quantum cryptography (free)	§1 and §3 from researchers in the field , including candid assessment of the assumptions.
Open Quantum Safe (free)	§3 and §4 as working code — hybrid TLS you can actually deploy and measure.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State precisely what Shor's and Grover's algorithms each do**, and the response to each.
- 2 **Explain why AES-256 needs nothing** while RSA needs replacement.
- 3 **Classify your own data** by required confidentiality lifetime.

- 4 **Identify which of it is affected** by harvest-now-decrypt-later, and which is not.
- 5 **Inventory every use of public-key cryptography** in a system you operate.
- 6 **Measure ML-KEM against X25519** for key size, ciphertext size, and time.
- 7 **Compute the added bytes per handshake** and what that costs on a constrained link.
- 8 **Enable a hybrid key exchange** somewhere and verify it negotiated.
- 9 **Audit one stored format** for hardcoded key sizes and missing algorithm identifiers.
- 10 **Write your proportionate plan**, with what you are explicitly not doing and why.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does Shor's algorithm break, and what does Grover's do?
- 2 Why is this a public-key problem rather than a cryptography problem?
- 3 Which primitives need nothing at all?
- 4 State the harvest-now-decrypt-later argument and what it makes this into.
- 5 What is the honest position on timing?
- 6 Why is key exchange more urgent than signatures?
- 7 Name the standardised schemes and state their cost.
- 8 Why deploy hybrid rather than replacing?
- 9 Give the four steps of a proportionate response.
- 10 Give five things not to do.